CCS335 – CLOUD COMPUTING LAB

B.Tech – Fifth Semester | Regulations 2021

Practical End Semester Examination Preparation

Answer format	Aim • Procedure • Program • Output • Result
Procedure length	At least 8 numbered steps per experiment
Source	CCS335 question paper – November/December 2026

This document covers every experiment listed in the uploaded question paper. The paper lists 15 questions (with non-consecutive question numbers), not 13. All 15 listed experiments are included below.

EXPERIMENT 1: Install a C Compiler in a Virtual Machine and Execute a Fibonacci Series Program Using C

1. Aim

To install a C compiler in a virtual machine and execute a C program to generate the Fibonacci series.

2. Procedure

1. Open virtualization software such as Oracle VirtualBox or VMware Workstation.
2. Start a virtual machine with an installed Linux operating system.
3. Open the Terminal application inside the Linux virtual machine.
4. Update the package repository using the command shown below.
5. Install the GCC compiler.
6. Verify the compiler installation by checking its version.
7. Create a C source file named fibonacci.c using a text editor.
8. Type the Fibonacci series program and save the file.
9. Compile the program using GCC to generate an executable file.
10. Run the executable, enter the number of terms and observe the series.

3. Program

Install and verify GCC:

```
sudo apt update
sudo apt install gcc -y
gcc --version
```

Program – fibonacci.c:

```
#include <stdio.h>

int main()
{
    int n, a = 0, b = 1, c, i;

    printf("Enter the number of terms: ");
    scanf("%d", &n);

    printf("Fibonacci Series:\n");

    for(i = 1; i <= n; i++)
    {
        printf("%d ", a);
        c = a + b;
        a = b;
        b = c;
    }

    return 0;
}
```

Compile and execute:

```
gcc fibonacci.c -o fibonacci
./fibonacci
```

4. Output of Program

```
Enter the number of terms: 10

Fibonacci Series:
0 1 1 2 3 5 8 13 21 34
```

5. Result

Thus, GCC was successfully installed in the virtual machine, and the C program to generate the Fibonacci series was compiled and executed successfully.

EXPERIMENT 2: Write a Google App Engine Program to Generate Nth Odd Numbers and Deploy It to Google Cloud

1. Aim

To develop a Python web application using Google App Engine to generate the first N odd numbers and deploy it to Google Cloud.

2. Procedure

1. Open Google Cloud Console at https://console.cloud.google.com/ and sign in.
2. Create a new Google Cloud project and select it.
3. Enable billing for the project if required by the account or deployment configuration.
4. Install Python and the Google Cloud CLI on the local computer.
5. Open a terminal and initialize the CLI using gcloud init; sign in and select the project.
6. Create a project directory named OddNumbers.
7. Inside the directory, create main.py and enter the Flask application shown below.
8. Create requirements.txt containing the Flask dependency.
9. Create app.yaml and specify the supported App Engine Python runtime.
10. Deploy the application using gcloud app deploy.
11. After deployment completes, open the application using gcloud app browse.
12. Enter a value for N and verify the generated odd numbers.

3. Program

File – main.py:

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/")
def odd_numbers():
    n = request.args.get("n")

    if n is None:
        return '''
        <h2>Generate Odd Numbers</h2>
        <form>
          Enter N:
          <input type="number" name="n" min="1" required>
          <input type="submit" value="Generate">
        </form>
        '''

    try:
        n = int(n)
        if n < 1 or n > 1000:
            return "Enter a number between 1 and 1000"

        result = [2 * i + 1 for i in range(n)]
        return f"<h2>First {n} Odd Numbers</h2>{result}"
    except ValueError:
        return "Enter a valid integer"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

File – requirements.txt:

```
Flask
```

File – app.yaml:

```
runtime: python313
```

Deployment commands:

```
gcloud init
gcloud app deploy
gcloud app browse
```

4. Output of Program

```
Browser input: N = 5

First 5 Odd Numbers:
[1, 3, 5, 7, 9]
```

Note: Keep main.py, requirements.txt and app.yaml in the same application directory. The runtime must be supported by the installed App Engine environment.

5. Result

Thus, the Python web application to generate the first N odd numbers was developed and deployed using Google App Engine.

EXPERIMENT 3: Create a Virtual Machine Using a Hypervisor and Install Linux

1. Aim

To create a virtual machine using a hypervisor and install a Linux operating system on it.

2. Procedure

1. Download and install Oracle VirtualBox or VMware Workstation on the host computer.
2. Download a Linux ISO image, such as Ubuntu, from its official website.
3. Open VirtualBox and click the New button.
4. Enter a virtual machine name such as Ubuntu and select Linux as the operating system type.
5. Allocate RAM according to the host computer's available memory; 4 GB is a typical lab allocation.
6. Create a virtual hard disk; allocate approximately 25 GB or more if available.
7. Attach the downloaded Linux ISO image to the virtual optical drive.
8. Start the virtual machine and select the Install Ubuntu option.
9. Choose the language, keyboard layout and installation type.
10. Configure the disk installation option and create a username and password.
11. Wait for the operating system installation to complete.
12. Restart the virtual machine, detach the ISO if prompted and log in using the created account.

3. Program / Commands

Not required. This experiment is performed using the specified software tools and commands.

4. Output of Program

```
VirtualBox displays the installed Linux desktop inside the virtual machine.
```

5. Result

Thus, a virtual machine was created using a hypervisor, and the Linux operating system was successfully installed and started.

EXPERIMENT 4: Set Up a One-Node Hadoop Cluster

1. Aim

To install and configure Hadoop in pseudo-distributed mode and establish a single-node Hadoop cluster.

2. Procedure

1. Install Ubuntu Linux and open the Terminal.
2. Update the package repository using sudo apt update.
3. Install a compatible Java Development Kit and verify it using java -version.
4. Install the OpenSSH server package.
5. Generate an SSH key pair and configure passwordless SSH access to localhost.
6. Download a Hadoop binary distribution from the Apache Hadoop website and extract it.
7. Set HADOOP_HOME, JAVA_HOME and the Hadoop bin/sbin directories in the shell environment.
8. Reload the environment configuration and verify Hadoop using hadoop version.
9. Edit etc/hadoop/core-site.xml to set the default filesystem to hdfs://localhost:9000.
10. Edit etc/hadoop/hdfs-site.xml and set dfs.replication to 1 for a single-node setup.
11. Configure mapred-site.xml to use the YARN framework.
12. Configure yarn-site.xml to enable the MapReduce shuffle service.
13. Format the NameNode once using hdfs namenode -format.
14. Start HDFS using start-dfs.sh and start YARN using start-yarn.sh.
15. Run jps and verify that the NameNode, DataNode, SecondaryNameNode, ResourceManager and NodeManager processes are running.

3. Program

Install packages and configure SSH:

```
sudo apt update
sudo apt install openjdk-11-jdk openssh-server -y
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
```

Example environment variables (add to ~/.bashrc):

```
export HADOOP_HOME=$HOME/hadoop-3.3.6
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin

source ~/.bashrc
```

core-site.xml:

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

hdfs-site.xml:

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
</configuration>
```

mapred-site.xml:

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

yarn-site.xml:

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

Start and verify:

```
hdfs namenode -format
start-dfs.sh
start-yarn.sh
jps
```

4. Output of Program

```
Sample jps output:
NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
Jps

NameNode web interface: http://localhost:9870
ResourceManager web interface: http://localhost:8088
```

Note: The Java path and Hadoop version may differ on the lab computer. Update JAVA_HOME and configuration paths accordingly. Format the NameNode only when initializing a fresh cluster.

5. Result

Thus, Hadoop was installed and configured in pseudo-distributed mode, and a single-node Hadoop cluster was established.

EXPERIMENT 5: Simulate a Cloud Scenario Using CloudSim and Run a Custom Scheduling Algorithm

1. Aim

To simulate a cloud computing environment using CloudSim and demonstrate a custom Shortest Cloudlet First (SCF) scheduling order.

2. Procedure

1. Install a compatible Java Development Kit.
2. Download the CloudSim toolkit and extract the package.
3. Open Eclipse or another Java IDE and create a project named CloudSimSimulation.
4. Add the CloudSim library files and required dependencies to the project build path.
5. Initialize the CloudSim simulation environment in the Java program.
6. Create a Datacenter with hosts, processing elements, memory, storage and bandwidth.
7. Create a DatacenterBroker to manage virtual machines and cloudlets.
8. Create virtual machines with suitable MIPS, RAM, bandwidth and storage values.
9. Create multiple cloudlets with different execution lengths.
10. Implement the custom Shortest Cloudlet First logic by sorting cloudlets in ascending order of length.
11. Submit the virtual machines and the sorted cloudlet list to the broker.
12. Start the CloudSim simulation, retrieve the cloudlet execution details and display the results.
13. Stop the simulation and inspect the scheduling output.

3. Program

Core SCF scheduling demonstration – SCFScheduler.java:

```
import java.util.Arrays;

public class SCFScheduler {
    public static void main(String[] args) {
        int[] cloudlets = {4000, 1000, 3000, 2000};

        System.out.println("Before Scheduling:");
        for (int c : cloudlets) {
            System.out.println(c);
        }

        // Sort cloudlets in ascending order of length
        Arrays.sort(cloudlets);

        System.out.println("\nAfter SCF Scheduling:");
        for (int c : cloudlets) {
            System.out.println("Cloudlet Length: " + c);
        }
    }
}
```

Compile and execute:

```
javac SCFScheduler.java
java SCFScheduler
```

4. Output of Program

```
Before Scheduling:
4000
1000
3000
```

```
2000

After SCF Scheduling:
Cloudlet Length: 1000
Cloudlet Length: 2000
Cloudlet Length: 3000
Cloudlet Length: 4000
```

Note: The code above demonstrates the SCF sorting logic; it is not, by itself, a complete CloudSim datacenter simulation. CloudSim APIs vary by version.

5. Result

Thus, the Shortest Cloudlet First scheduling order was demonstrated by sorting cloudlets according to execution length. For a complete CloudSim experiment, this order must be connected to the CloudSim broker and executed within the configured simulation.

EXPERIMENT 6: Install Windows in VirtualBox and Find the Largest Element in an Array Using C

1. Aim

To install Windows in Oracle VirtualBox and execute a C program to find the largest element in a given array.

2. Procedure

1. Install Oracle VirtualBox on the host computer.
2. Download a Windows ISO image from the official Microsoft website.
3. Open VirtualBox and click New to create a virtual machine.
4. Enter the virtual machine name and select Microsoft Windows as the operating system type.
5. Allocate RAM and create a virtual hard disk with sufficient storage.
6. Attach the Windows ISO image and start the virtual machine.
7. Follow the Windows setup instructions and complete the operating system installation.
8. Install a C compiler such as MinGW-w64 or an IDE bundled with a compiler.
9. Open the compiler terminal and create a file named largest.c.
10. Enter the C program to read array elements and find the maximum value.
11. Compile the program using GCC and execute the generated program.
12. Enter the array size and elements, then verify the largest value.

3. Program

Program – largest.c:

```
#include <stdio.h>

int main()
{
    int a[100], n, i, max;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    if (n < 1 || n > 100) {
        printf("Invalid array size");
        return 1;
    }

    printf("Enter array elements:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    max = a[0];

    for (i = 1; i < n; i++) {
        if (a[i] > max) {
            max = a[i];
        }
    }

    printf("Largest element = %d\n", max);
    return 0;
}
```

Compile and execute in the compiler environment:

```
gcc largest.c -o largest.exe
```

```
largest.exe
```

4. Output of Program

```
Enter number of elements: 5
Enter array elements:
10
25
45
12
30
Largest element = 45
```

5. Result

Thus, Windows was installed in VirtualBox, and the C program to find the largest element in an array was compiled and executed successfully.

EXPERIMENT 7: Install Google App Engine and Create a Python Temperature Converter Web Application

1. Aim

To develop and deploy a Python-based temperature converter web application using Google App Engine.

2. Procedure

1. Open Google Cloud Console and sign in with a Google account.
2. Create a new Google Cloud project and select it.
3. Enable billing if required for the project.
4. Install Python and the Google Cloud CLI on the local computer.
5. Open a terminal and initialize the CLI using gcloud init.
6. Sign in and select the Google Cloud project.
7. Create a directory named TemperatureConverter.
8. Create main.py and enter the Flask temperature-conversion application.
9. Create requirements.txt and add Flask as a dependency.
10. Create app.yaml and specify the supported App Engine runtime.
11. Deploy the application using gcloud app deploy.
12. Open the deployed application using gcloud app browse.
13. Enter a Celsius temperature and verify the Fahrenheit result.

3. Program

File – main.py:

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/")
def home():
    c = request.args.get("c")

    if c is None:
        return '''
        <h2>Temperature Converter</h2>
        <form>
          Enter Celsius:
          <input type="number" step="any" name="c" required>
          <input type="submit" value="Convert">
        </form>
        '''

    try:
        celsius = float(c)
        fahrenheit = (celsius * 9 / 5) + 32
        return (f"<h2>Temperature Conversion</h2>"
                f"Celsius: {celsius}<br>"
                f"Fahrenheit: {fahrenheit:.2f}")
    except ValueError:
        return "Enter a valid temperature"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

File – requirements.txt:

```
Flask
```

File – app.yaml:

```
runtime: python313
```

Deployment commands:

```
gcloud init
gcloud app deploy
gcloud app browse
```

4. Output of Program

```
Input: Celsius = 25

Output:
Celsius: 25.0
Fahrenheit: 77.00
```

5. Result

Thus, a temperature converter web application was developed using Python and deployed using Google App Engine.

EXPERIMENT 8: Create a Virtual Machine and Transfer Files Between Host and Guest Using Drag and Drop

1. Aim

To create a virtual machine using VirtualBox and transfer files between the host and guest operating systems using drag and drop.

2. Procedure

1. Download and install Oracle VirtualBox on the host system.
2. Create a new virtual machine and select the required guest operating system.
3. Allocate memory and virtual disk space, then install the guest operating system.
4. Start the guest operating system and log in.
5. From the VirtualBox menu, select Devices and then Insert Guest Additions CD Image.
6. Install Guest Additions inside the guest operating system and restart the virtual machine.
7. Open the virtual machine Settings and select General, then Advanced.
8. Set Shared Clipboard to Bidirectional.
9. Set Drag and Drop to Bidirectional and save the settings.
10. Start or restart the guest machine and open its file manager.
11. Select a file on the host machine and drag it into the guest machine.
12. Verify that the file appears in the guest machine.
13. Repeat the operation in the reverse direction to transfer a file from guest to host.

3. Program / Commands

Not required. This experiment is performed using the specified software tools and commands.

4. Output of Program

```
Files are visible in the destination operating system after being dragged from the source operating system.
```

Note: Guest Additions must be installed, and drag-and-drop must be enabled. Support can depend on the host/guest operating-system combination.

5. Result

Thus, the virtual machine was created using VirtualBox, and files were transferred between the host and guest operating systems using the drag-and-drop feature.

EXPERIMENT 9: Install a Hadoop Single-Node Cluster on Windows and Execute Word Count

1. Aim

To install and configure a single-node Hadoop environment on Windows and execute the Word Count MapReduce application.

2. Procedure

1. Install a compatible Java Development Kit on Windows.
2. Set JAVA_HOME to the JDK installation directory.
3. Download a Hadoop binary distribution and extract it, for example, to C:\\hadoop.
4. Set HADOOP_HOME and add the Hadoop bin directory to the system PATH.
5. Configure core-site.xml, hdfs-site.xml, mapred-site.xml and yarn-site.xml.
6. Configure Windows-compatible Hadoop native binaries, including a version-compatible winutils.exe where required.
7. Create the required NameNode and DataNode storage directories.
8. Format the NameNode from the Hadoop command prompt.
9. Start HDFS and YARN using the Windows command scripts supplied by the distribution.
10. Create an HDFS input directory using hdfs dfs -mkdir -p /input.
11. Create a text file named input.txt and enter sample words.
12. Upload the file to HDFS using hdfs dfs -put input.txt /input.
13. Run the Hadoop MapReduce examples JAR with the wordcount command.
14. Display the output using hdfs dfs -cat /output/part-r-00000.

3. Program

Sample input file – input.txt:

```
hello world
hello hadoop
hadoop is useful
```

Hadoop Word Count command (adjust the JAR version/path as installed):

```
hdfs dfs -mkdir -p /input
hdfs dfs -put input.txt /input

hadoop jar %HADOOP_HOME%\share\hadoop\mapreduce\hadoop-mapreduce-examples-3.3.6.jar wordcount /input /output

hdfs dfs -cat /output/part-r-00000
```

4. Output of Program

```
hadoop  2
hello   2
is      1
useful  1
world   1
```

Note: Windows Hadoop setup requires native binaries compatible with the exact Hadoop version. The example JAR path and version must match the installed distribution.

5. Result

Thus, Hadoop was configured in single-node mode on Windows, and the Word Count MapReduce application was executed successfully.

EXPERIMENT 10: Install Google App Engine SDK on Windows and Run a Hello World Application

1. Aim

To install the Google Cloud development tools on Microsoft Windows and create and execute a simple Hello World application using Google App Engine.

2. Procedure

1. Open a web browser and visit the Google Cloud website.
2. Download the Google Cloud CLI installer for Windows.
3. Run the installer and complete the installation steps.
4. Install Python if it is not already available on the system.
5. Open the Google Cloud SDK Shell or Command Prompt.
6. Initialize the CLI using gcloud init.
7. Sign in using a Google account and select the required Google Cloud project.
8. Create a directory named HelloWorld.
9. Inside the directory, create a Python file named main.py.
10. Enter the Flask program that displays the Hello World message.
11. Create requirements.txt containing Flask.
12. Create app.yaml and specify the supported App Engine runtime.
13. Deploy the application using gcloud app deploy.
14. After deployment, run gcloud app browse and verify the message in the browser.

3. Program

File – main.py:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello():
    return "<h1>Hello World!</h1>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

File – requirements.txt:

```
Flask
```

File – app.yaml:

```
runtime: python313
```

Deployment commands:

```
gcloud init
gcloud app deploy
gcloud app browse
```

4. Output of Program

```
Browser output:

Hello World!
```

```
Application deployed successfully.
```

5. Result

Thus, the Google Cloud development tools were installed on Windows, and a simple Hello World application was created and deployed using Google App Engine.

EXPERIMENT 11: Create and Execute Your First Container Using Docker

1. Aim

To use Docker to download an image, create a container and execute commands inside the container.

2. Procedure

1. Visit the official Docker website and install Docker Desktop or Docker Engine for the operating system.
2. Start the Docker service or Docker Desktop.
3. Open a terminal or command prompt.
4. Verify the Docker installation using docker --version.
5. Check the Docker service using docker info.
6. Download the official Ubuntu image from Docker Hub using docker pull ubuntu.
7. Verify the downloaded image using docker images.
8. Create and start a container interactively using docker run -it --name mycontainer ubuntu bash.
9. Observe the container shell prompt and execute a command such as echo Hello Docker.
10. Exit the container using the exit command.
11. List all containers, including stopped containers, using docker ps -a.
12. Start the stopped container again using docker start -ai mycontainer.

3. Program

Docker commands:

```
docker --version
docker info
docker pull ubuntu
docker images
docker run -it --name mycontainer ubuntu bash

# Run inside the container:
echo "Hello Docker"
exit

# On the host:
docker ps -a
docker start -ai mycontainer
```

4. Output of Program

```
docker pull ubuntu
Status: Downloaded newer image for ubuntu:latest

$ docker run -it --name mycontainer ubuntu bash
root@container:/# echo "Hello Docker"
Hello Docker
root@container:/# exit
```

5. Result

Thus, Docker was used to download an Ubuntu image, create a container and execute commands inside it successfully.

EXPERIMENT 12: Run a Container from Docker Hub

1. Aim

To download a Docker image from Docker Hub and run a container using the downloaded image.

2. Procedure

1. Open a web browser and visit https://hub.docker.com/.
2. Install Docker Desktop or Docker Engine and start the Docker service.
3. Open a terminal or command prompt.
4. Verify the Docker installation using docker --version.
5. Search for the Nginx image using docker search nginx.
6. Download the official Nginx image using docker pull nginx.
7. Verify the downloaded image using docker images.
8. Run an Nginx container in detached mode and map host port 8080 to container port 80.
9. Check the running container using docker ps.
10. Open a web browser and enter http://localhost:8080.
11. Verify that the Nginx welcome page is displayed.
12. Stop the container using docker stop mynginx.
13. Remove the container if required using docker rm mynginx.

3. Program

Docker commands:

```
docker --version
docker search nginx
docker pull nginx
docker images
docker run -d --name mynginx -p 8080:80 nginx
docker ps

# Open http://localhost:8080 in a browser
docker stop mynginx
docker rm mynginx
```

4. Output of Program

```
Sample terminal output:

$ docker run -d --name mynginx -p 8080:80 nginx
<container-id>

$ docker ps
CONTAINER ID   IMAGE   STATUS
xxxxxxxx       nginx   Up

Browser: Welcome to nginx!
```

5. Result

Thus, an Nginx image was downloaded from Docker Hub, and a container was created and executed successfully.

EXPERIMENT 13: Transfer Files from One Virtual Machine to Another Virtual Machine

1. Aim

To establish communication between two virtual machines and transfer a file using SSH and SCP.

2. Procedure

1. Install VirtualBox or VMware Workstation on the host computer.
2. Create two virtual machines named VM1 and VM2.
3. Install Linux on both virtual machines.
4. Configure both virtual machines to use the same reachable network, such as a host-only network or bridged network.
5. Start both virtual machines and log in.
6. Install the OpenSSH server on both virtual machines.
7. Find the IP address of VM1 using hostname -I.
8. Find the IP address of VM2 using hostname -I.
9. From VM1, test connectivity to VM2 using ping and its IP address.
10. Create a sample file in VM1 using echo and redirect the text into sample.txt.
11. Use the scp command on VM1 to copy sample.txt to the home directory of the user on VM2.
12. Enter the VM2 user's password when prompted, if password authentication is enabled.
13. Log in to VM2 and use ls to verify that sample.txt has arrived.
14. Display the file contents using cat sample.txt.

3. Program

Install SSH and identify addresses:

```
sudo apt update
sudo apt install openssh-server -y
hostname -I
ping <VM2-IP>
```

Create and transfer a file from VM1:

```
echo "File Transfer Successful" > sample.txt
scp sample.txt username@<VM2-IP>:/home/username/
```

Verify on VM2:

```
ls
cat sample.txt
```

4. Output of Program

```
VM1:
$ scp sample.txt user@192.168.56.102:/home/user/
sample.txt 100% transferred

VM2:
$ ls
sample.txt

$ cat sample.txt
File Transfer Successful
```

Note: Replace the username and example IP address with the actual VM2 username and IP address. Both virtual machines must be able to reach each other over the selected network.

5. Result

Thus, two virtual machines were connected through a network, and a file was transferred from VM1 to VM2 using SCP.

EXPERIMENT 14: Install a C Compiler in a Virtual Machine and Execute a File Read/Write Program

1. Aim

To install a C compiler in a virtual machine and execute a C program to perform file write and read operations.

2. Procedure

1. Open Oracle VirtualBox or VMware Workstation.
2. Start a virtual machine with an installed Linux operating system.
3. Open the Terminal application.
4. Update the package repository using sudo apt update.
5. Install GCC using sudo apt install gcc -y.
6. Verify the compiler installation using gcc --version.
7. Create a source file named fileoperation.c using a text editor.
8. Enter the C program to write data into a file and read it back.
9. Save the file and compile it using GCC.
10. Execute the compiled program.
11. Enter the text to be written into the file.
12. Verify the success message displayed after the write operation.
13. Observe the file contents displayed by the read operation.
14. Use ls or a file manager to confirm that sample.txt has been created.

3. Program

Install GCC:

```
sudo apt update
sudo apt install gcc -y
gcc --version
```

Program – fileoperation.c:

```
#include <stdio.h>

int main()
{
    FILE *fp;
    char text[100];

    // Write operation
    fp = fopen("sample.txt", "w");
    if (fp == NULL) {
        printf("File cannot be opened");
        return 1;
    }

    printf("Enter text to write into file: ");
    fgets(text, sizeof(text), stdin);
    fprintf(fp, "%s", text);
    fclose(fp);

    printf("\nData written successfully.\n");

    // Read operation
    fp = fopen("sample.txt", "r");
    if (fp == NULL) {
```

```
        printf("File cannot be opened");
        return 1;
    }

    printf("\nContents of the file:\n");
    while (fgets(text, sizeof(text), fp) != NULL) {
        printf("%s", text);
    }

    fclose(fp);
    return 0;
}
```

Compile and execute:

```
gcc fileoperation.c -o fileoperation
./fileoperation
```

4. Output of Program

```
Enter text to write into file:
Cloud Computing Lab

Data written successfully.

Contents of the file:
Cloud Computing Lab

Created file: sample.txt
```

5. Result

Thus, GCC was installed in the virtual machine, and the C program to perform file read and write operations was compiled and executed successfully.

EXPERIMENT 15: Write a Google App Engine Program to Display the Nth Largest Number from a List and Deploy It to Google Cloud

1. Aim

To develop a Python web application using Google App Engine to find and display the Nth largest number from a given list and deploy it to Google Cloud.

2. Procedure

1. Open Google Cloud Console and sign in.
2. Create a new Google Cloud project and select it.
3. Enable billing if required.
4. Install Python and Google Cloud CLI on the local computer.
5. Initialize the CLI using gcloud init and select the project.
6. Create a directory named NthLargest.
7. Inside the directory, create main.py.
8. Write a Flask application that accepts a comma-separated list and the value N.
9. Convert the entered values into integers and validate the value of N.
10. Sort the list in descending order.
11. Select the element at index N - 1 as the Nth largest element.
12. Create requirements.txt containing Flask.
13. Create app.yaml and specify the supported App Engine runtime.
14. Deploy the application using gcloud app deploy.
15. Open the application using gcloud app browse and test it with sample values.

3. Program

File – main.py:

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/")
def largest():
    numbers = request.args.get("numbers")
    n = request.args.get("n")

    if numbers is None or n is None:
        return '''
        <h2>Nth Largest Number</h2>
        <form>
          Enter numbers (comma separated):
          <input name="numbers" required><br><br>
          Enter N:
          <input type="number" name="n" min="1" required><br><br>
          <input type="submit" value="Find">
        </form>
        '''

    try:
        arr = [int(x.strip()) for x in numbers.split(",")]
        n = int(n)

        if n < 1 or n > len(arr):
```

```
            return "Invalid N value"

        arr.sort(reverse=True)
        result = arr[n - 1]
        return f"<h2>{n}th Largest Number = {result}</h2>"

    except ValueError:
        return "Enter valid numbers"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

File – requirements.txt:

```
Flask
```

File – app.yaml:

```
runtime: python313
```

Deployment commands:

```
gcloud init
gcloud app deploy
gcloud app browse
```

4. Output of Program

```
Input numbers: 10, 50, 30, 20, 40
N = 2

Output:
2nd Largest Number = 40
```

Note: This program treats repeated values as separate list elements. For example, [50, 50, 40] has 50 as both the first and second largest elements under this convention.

5. Result

Thus, a Python web application to find the Nth largest number from a given list was developed and deployed using Google App Engine.

QUICK REVISION – IMPORTANT COMMANDS

Use this section to revise the commands and tools commonly required during the practical examination.

Topic	Important commands / tools
Linux C compiler	sudo apt install gcc -y | gcc file.c -o program | ./program
Hadoop	hdfs namenode -format | start-dfs.sh | start-yarn.sh | jps
Google App Engine	gcloud init | gcloud app deploy | gcloud app browse
Docker	docker pull ubuntu | docker images | docker run -it ubuntu bash | docker ps -a
VM-to-VM transfer	scp sample.txt user@<VM2-IP>:/home/user/
Java / Hadoop check	java -version | hadoop version | jps
HDFS Word Count	hdfs dfs -put input.txt /input | hadoop jar <examples.jar> wordcount /input /output

Exam reminders:

- For App Engine experiments, keep main.py, requirements.txt and app.yaml in the same application directory.
- For Hadoop, configure Java and Hadoop environment variables before starting the services.
- For Docker, an image is a template; a container is a running or stopped instance created from an image.
- For VirtualBox drag-and-drop, install Guest Additions and enable the required bidirectional setting.
- For CloudSim, the SCF sorting demonstration must be integrated with the broker and a real CloudSim simulation for a complete simulation result.